CASA Evidence — OAuth 2.0 Integration and Flow

Question: *If the application uses an OAuth 2.0 integration, provide a written description along with relevant evidence (screenshots, source code, or other documentation) detailing which OAuth 2.0 flow is used.*

Application: Provenance (Next.js App Router), using Supabase Auth (GoTrue) as its authentication provider, with federated sign-in via Google and Apple.

---

1. Summary

Provenance uses OAuth 2.0 **only** for third-party ("social") sign-in — it is not used for any other integration (no third-party API consumes an OAuth token issued by this app, and no other outbound integration in this codebase uses OAuth). The two configured identity providers are **Google** and **Apple ("Sign in with Apple")**:

```
const oAuthProviders = ['google', 'apple'] as const;

console.log('[Auth] providers configured', {
  password: passwordEnabled,
  magicLink: magicLinkEnabled,
  oAuth: oAuthProviders,
});
```

The flow used is **OAuth 2.0 Authorization Code Grant with Proof Key for Code Exchange (PKCE)** — RFC 6749 (Authorization Code Grant) + RFC 7636 (PKCE) — never the deprecated Implicit Grant. This is enforced by the auth library itself (`@supabase/ssr`), not something the application had to opt into or could accidentally disable.

The application does not act as its own OAuth Authorization Server or Resource Server against Google/Apple directly. Instead, **Supabase Auth (GoTrue) is the OAuth client of record with each identity provider** and acts as a broker: it performs the browser-facing Authorization Code exchange with Google/Apple using its own

confidential client credentials (Client ID + Client Secret, held only in the Supabase project dashboard — never in this application's codebase or environment variables), and then issues Provenance its **own** PKCE-protected authorization code for a Supabase session. The application only ever performs the second, PKCE-protected hop.

---

2. Evidence: the flow is PKCE, not Implicit, and is not optional

The Supabase SSR client library used throughout the app (`@supabase/ssr`) hardcodes `flowType: "pkce"` in both the browser and server client constructors — it cannot be silently downgraded to the Implicit flow, which is important because the Implicit flow returns tokens directly in a URL fragment that a server-rendered app could never safely read.

Official Supabase documentation confirms this design choice explicitly:

> "In the `@supabase/ssr` package, Supabase clients are initiated to
> use the PKCE flow by default. … At present, PKCE is supported on
> the Magic Link, OAuth, Sign Up, and Password Recovery routes."
> — Supabase: Server-Side Auth advanced guide

And the library's own source (`@supabase/ssr`, `createBrowserClient.ts` / `createServerClient.ts`) sets this unconditionally, after spreading any caller-supplied `auth` options, so an application cannot accidentally override it:

```
auth: {
  ...options?.auth,
  flowType: "pkce",           // always PKCE — not caller-overridable
  autoRefreshToken: ...,
  detectSessionInUrl: ...,
  persistSession: true,
  storage,
},
```

Provenance's own client factories all go through this library and inherit `flowType: "pkce"` unmodified:

```
export function getSupabaseBrowserClient<GenericSchema = Database>() {
```

```
  const keys = getSupabaseClientKeys();

  return createBrowserClient<GenericSchema>(keys.url, keys.anonKey, {
    cookieOptions: getHardenedCookieOptions(),
  });
}

export function getSupabaseServerClient<GenericSchema = Database>() {
  const keys = getSupabaseClientKeys();

  return createServerClient<GenericSchema>(keys.url, keys.anonKey, {
    cookieOptions: getHardenedCookieOptions(),
    cookies: { ... },
  });
}
```

`createMiddlewareClient()` (used in `src/middleware.ts` for session refresh on every request) is built the same way, via the same `createServerClient` call, so the same PKCE configuration applies across the browser client, server client, and middleware client consistently.

---

3. Evidence: initiating the flow (browser → Supabase authorize endpoint)

The sign-in UI renders one button per configured provider and calls `supabase.auth.signInWithOAuth()` on click:

```
function SignInPage() {
  ...
  <SignInMethodsContainer paths={paths} providers={authConfig.providers} />

const redirectTo = [origin, redirectPath].join('');
const scopesOpts = OAUTH_SCOPES[provider] ?? {};

const credentials = {
  provider,
  options: {
    shouldCreateUser: props.shouldCreateUser,
    redirectTo,
    ...scopesOpts,
  },
};

return onSignInWithProvider(() =>
  signInWithProviderMutation.mutateAsync(credentials),
);

const mutationFn = async (credentials: SignInWithOAuthCredentials) => {
  const response = await client.auth.signInWithOAuth(credentials);

  if (response.error) {
    throw response.error.message;
  }
```

```
  return response.data;
};
```

Because `flowType: "pkce"` is set on the client (§2), calling `signInWithOAuth()` causes the Supabase JS SDK to, before redirecting the browser anywhere:

1. Generate a cryptographically random `code_verifier`.
2. Derive a `code_challenge` (`S256` hash of the verifier) per RFC 7636.
1. Persist the `code_verifier` in a first-party cookie (via the app's cookie storage adapter — see `getHardenedCookieOptions()` below) so it can be presented again once the flow completes.
1. Redirect the browser to Supabase's `/auth/v1/authorize` endpoint with the provider name, the app's own `redirectTo` (`/auth/callback?next=...`), and the `code_challenge`.

The code contains a specific, deliberate host-consistency guard for this exact PKCE cookie, which is itself strong evidence that the code authors understood and were actively defending the PKCE mechanics:

```
/**
 * @name APEX_TO_WWW_HOST
 * @description
 * `provenance.guru` (apex) 307s to `www.provenance.guru` at the Vercel domain
 * level. Supabase's PKCE `code_verifier` cookie is host-only (no `domain`
 * attribute — see `getHardenedCookieOptions`), so if a user ever lands on
 * this page via the bare apex host, the cookie set here would not be visible
 * once Supabase's OAuth callback resolves back to the canonical `www` host,
 * breaking `exchangeCodeForSession` with a generic "Authentication Error".
 * Force the canonical host before starting the OAuth flow so the cookie and
 * the callback always agree on the same host.
 */
```

---

4. Evidence: Supabase ↔ identity-provider hop (Google/Apple as confidential client)

Supabase Auth is registered as the **OAuth client** with each identity provider — the app itself is never registered with Google or Apple directly, and never holds their client secrets. This is documented in the operational setup notes kept alongside the codebase:

```
### 1.3 Enable Google Provider in Supabase
```

```
1. In Supabase Dashboard, go to **Authentication** → **Providers**
2. Find **Google** in the list
3. Make sure it's **Enabled** (toggle should be ON)
4. If you need to configure Google OAuth credentials:
   - You'll need Client ID and Client Secret from Google Cloud Console
   - Enter them in the Google provider settings

### 2.4 Update Authorized Redirect URIs

In the same OAuth client configuration, find **Authorized redirect URIs**:

1. **Verify** this URI is already there (it should be):
```

https://your-project-id.supabase.co/auth/v1/callback

```
   - Replace `your-project-id` with your actual Supabase project ID
   - This is the Supabase callback URL, NOT your app URL
   - If it's not there, click **+ ADD URI** and add it

2. **Do NOT add** `https://provenance.guru/auth/callback` here
   - Google redirects to Supabase, not directly to your app
   - Supabase then redirects to your app
```

This confirms the two-hop broker architecture directly: Google's (and Apple's) Authorization Code is issued to **Supabase's** redirect URI, exchanged for tokens by **Supabase**, server-side, using credentials that never leave the Supabase project — the application's codebase and environment variables (`.env.example`, Vercel env vars) contain no Google/Apple OAuth client secret at all, confirming the principle-of-least-privilege posture already documented in `principle-of-least-privilege.md`: this application never has custody of the upstream IdP's confidential client credential.

---

5. Evidence: completing the flow (app's own PKCE code exchange)

After the upstream IdP hop, Supabase redirects the browser back to the application's own callback route with **its own** authorization code (distinct from Google/Apple's), e.g. `https://provenance.guru/auth/callback?code=...`:

```
export async function GET(request: NextRequest) {
  // Diagnostic logging (temporary): confirms whether magic-link/OAuth clicks
  // are reaching this route at all, and whether a `code` param is present
  // (PKCE flow) — helps distinguish "email link never arrives" (this never
  // logs) from "link arrives but exchange fails" (logs, then errors below).
  console.log('[Auth/Callback] request received', {
    host: request.headers.get('host'),
    hasCode: request.nextUrl.searchParams.has('code'),
```

```
    hasError: request.nextUrl.searchParams.has('error'),
    next: request.nextUrl.searchParams.get('next'),
  });
```

The route hands the code to a shared callback service, which calls `auth.exchangeCodeForSession(authCode)`:

```
async exchangeCodeForSession(
  request: Request,
  params: {
    redirectPath: string;
    errorPath?: string;
  },
): Promise<{ nextPath: string }> {
  const requestUrl = new URL(request.url);
  const searchParams = requestUrl.searchParams;
  const authCode = searchParams.get('code');
  ...
  if (authCode) {
    console.log('[Auth/Callback] exchangeCodeForSession starting', {
      authCodeLength: authCode.length,
      nextUrl,
    });

    try {
      const { error, data } =
        await this.client.auth.exchangeCodeForSession(authCode);
      ...
```

Internally, `exchangeCodeForSession()` reads back the `code_verifier` cookie set in §3, sends it to Supabase's token endpoint alongside the authorization code, and Supabase verifies `SHA256(code_verifier) == code_challenge` before minting a session — this is the PKCE verification step that stops a stolen/replayed authorization code from being redeemed by anyone other than the browser that started the flow. The service explicitly detects and reports this exact failure mode by name:

```
/**
 * Checks if the given error message indicates a PKCE verifier error.
 * Multiple Supabase error message variants are checked because the exact
 * wording has changed across Supabase versions and edge cases (e.g. empty
 * verifier vs verifier mismatch vs missing verifier).
 */
function isVerifierError(error: string) {
  const lower = error.toLowerCase();
  return (
    lower.includes('both auth code and code verifier should be non-empty') ||
    lower.includes('code verifier') ||
    lower.includes('pkce') ||
    lower.includes('code_verifier')
  );
}
```

On success, the resulting Supabase session (access + refresh JWT) is

written to the app's own first-party, hardened cookies:

```
export function getHardenedCookieOptions(): CookieOptionsWithName {
  return {
    path: '/',
    sameSite: 'lax',
    secure: process.env.NODE_ENV === 'production',
  };
}
```

The callback route then provisions the account if new

(`provisionNewUser`) and redirects the now-authenticated user onward —

with an explicit open-redirect guard on the client-supplied `next`

parameter, since it is otherwise attacker-influenceable input riding

along on the OAuth redirect chain:

```
// Guard against open redirects (CASA 5.1.2 / CWE-601): the `next` param on this
// route comes straight from the query string via @kit/supabase's auth callback
// service. The block above strips the host from full absolute URLs
// (e.g. `https://evil.com/x` -> `/x`), but protocol-relative URLs like
// `//evil.com/x` fail the `new URL(nextPath)` parse (no base), fall into the
// catch, and are used as-is. `new URL('//evil.com/x', origin)` then resolves
// to `https://evil.com/x` because a leading `//` is a network-path reference
// that takes over the host from `origin`. Reject anything that isn't a
// same-origin, single-leading-slash path before building the redirect.
if (!pathToRedirect.startsWith('/') || pathToRedirect.startsWith('//') || pathToRedirect.startsWith('/\\')) {
  console.warn('[Auth] Rejected unsafe post-login redirect target', { nextPath });
  pathToRedirect = '/artworks';
}
```

---

6. End-to-end sequence (Google, illustrative — Apple is identical apart from provider name)

1. User clicks **"Sign in with Google"** on `provenance.guru`.
2. Browser calls `supabase.auth.signInWithOAuth({ provider: 'google', options: { redirectTo: '.../auth/callback?next=...' } })`. The Supabase JS SDK generates the PKCE `code_verifier`/`code_challenge` pair and stores the verifier in a first-party cookie.
3. Browser is redirected to Supabase's `/auth/v1/authorize?provider=google&code_challenge=...`.
4. Supabase (as the registered OAuth client) redirects to Google's OAuth consent screen.
5. User authenticates and authorizes on Google's own domain.
6. Google issues an authorization code to **Supabase's** fixed redirect URI: `https://<project>.supabase.co/auth/v1/callback`.
7. Supabase exchanges Google's code for Google tokens server-side, using its own Client ID/Secret (never exposed to the app), and retrieves the user's Google profile.
8. Supabase mints its **own** authorization code for this login and redirects the browser to the app's `redirectTo`: `https://provenance.guru/auth/callback?code=...`.
9. The app's `route.ts` calls `exchangeCodeForSession(code)`, which pairs the code with the `code_verifier` cookie from step 2 and calls Supabase's token endpoint.

10 Supabase validates `SHA256(code_verifier) == code_challenge`, and if valid, returns a Supabase session (JWT access + refresh token).

11 The session is written to hardened, first-party cookies (`secure` in production, `SameSite=Lax`); `provisionNewUser()` runs for first-time sign-ins; the user is redirected into the app.

---

7. Why this flow (and not Implicit or Client Credentials)

1 **Implicit Grant** would return tokens directly in the URL fragment of the redirect back to the app. A server-rendered Next.js app cannot safely read a URL fragment on the server, and tokens in a URL are exposed to browser history, referrer headers, and any script on the page — which is exactly why `@supabase/ssr` refuses to allow it (§2) and why Supabase's own guidance calls out Implicit as the flow to move *away from* for SSR apps.

1 **Client Credentials Grant** is not applicable here — there is no machine-to-machine OAuth exchange with Google/Apple; every grant is tied to an interactive end-user login.

1 **Authorization Code + PKCE** is the correct choice for this architecture because it keeps the only genuinely confidential credential (the IdP client secret) inside Supabase's managed backend, while still protecting the app's own authorization-code hop against interception/replay via the `code_verifier`/`code_challenge` pair — without the app ever needing to hold a client secret of its own.

---

8. Conclusion

Provenance's OAuth 2.0 integration — federated sign-in via Google and Apple — uses the **Authorization Code Grant with PKCE** end to end, enforced unconditionally by the `@supabase/ssr` client library (`flowType: "pkce"`, not configurable to anything else) and completed by an explicit `exchangeCodeForSession()` call in `src/app/auth/callback/route.ts` that redeems the code against a

cookie-stored `code_verifier`. Supabase Auth acts as the confidential OAuth client with each upstream identity provider, so this application's own codebase and infrastructure never hold Google's or Apple's OAuth client secret — the only OAuth artifact the app itself ever handles is its own short-lived, PKCE-bound authorization code.